



Paradigmas de Programación

Clases teóricas

2do cuatrimestre de 2026

Departamento de Computación, FCEyN, UBA

2. Funcional 2

Esquemas de recursión

“Detrás de cada acontecimiento hay un truco de espejos.
Nada es, todo parece. Escóndase, si quiere. Espíe por
las ranuras. Alguien estará preparando otra ilusión. Las
diferencias entre las personas son las diferencias entre
las ilusiones que perciben.”
(Consejero, 121:6:33)

El Fondo del Pozo
Eduardo Abel Giménez





De dónde venimos

La clase pasada, al final

- Teníamos dos funciones que hacían “lo mismo” sobre listas
- Recuadramos lo que cambiaba, lo recortamos, le pusimos nombre

- Salió **map**

```
map f []      = []  
map f (x : xs) = f x : map f xs
```

- Con **filter** pasa lo mismo
- Hoy: ¿hasta dónde llega esa idea?

La pregunta de hoy

- Los esquemas **map** y **filter** son dos casos
- ¿Habrá un esquema **más general** del que los dos sean instancias?
- ¿Y uno para otros tipos de datos (no solo para listas)?
- Spoiler: sí, y la forma del esquema **está determinada por la forma del tipo**



Plegar una lista

Tres funciones

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Considerar estas tres funciones sobre listas

□ Las tres con recursión explícita:

```
todosCincos  :: [Int] -> Bool  
todosCincos []      = True  
todosCincos (n : ns) = n == 5 && todosCincos ns
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas []      = 0  
totalMonedas (p : ps) = cantMonedas p + totalMonedas ps
```

```
concat      :: [[a]] -> [a]  
concat []   = []  
concat (xs : xss) = xs ++ concat xss
```

■ Ninguna es un **map** ni un **filter**

■ ¿Qué tienen en común?

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 1

□ Recuadrar las diferencias

```
todosCincos  :: [Int] -> Bool  
todosCincos [] = True  
todosCincos (n : ns) = n == 5 && todosCincos ns
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas [] = 0  
totalMonedas (p : ps) = cantMonedas p + totalMonedas ps
```

```
concat      :: [[a]] -> [a]  
concat []   = []  
concat (xs : xss) = xs ++ concat xss
```

■ Cambian **dos** cosas: el valor del caso base y cómo se combina

■ Pero se recuadró **todo**: no quedó nada afuera, y así no se parecen en nada

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Técnica extendida

- Redondear lo que el recuadro toma de afuera

```
todosCincos :: [Int] -> Bool  
todosCincos [] = True  
todosCincos (n : ns) = n == 5 && todosCincos ns
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas [] = 0  
totalMonedas (p : ps) = cantMonedas p + totalMonedas ps
```

```
concat :: [[a]] -> [a]  
concat [] = []  
concat (xs : xss) = xs ++ concat xss
```

- Adentro del recuadro hay dos cosas que no son tuyas: el elemento y la recursión

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Técnica extendida

- Dejar un agujero por cada cosa que salió

```
todosCincos :: [Int] -> Bool  
todosCincos [] = True  
todosCincos (n : ns) = ( == 5 && ) n (todosCincos ns)
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas [] = 0  
totalMonedas (p : ps) = (cantMonedas + ) p (totalMonedas ps)
```

```
concat :: [[a]] -> [a]  
concat [] = []  
concat (xs : xss) = ( ++ ) xs (concat xss)
```

- Salieron las dos, y el recuadro quedó **aplicado** a ellas

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Técnica extendida

- Hacer lugar adelante, para lo que va a nombrarlos

```
todosCincos  :: [Int] -> Bool  
todosCincos [] = True  
todosCincos (n : ns) = (        == 5 &&        ) n (todosCincos ns)  
  
totalMonedas :: [Persona] -> Int  
totalMonedas [] = 0  
totalMonedas (p : ps) = ( cantMonedas        +        ) p (totalMonedas ps)  
  
concat :: [[a]] -> [a]  
concat [] = []  
concat (xs : xss) = (       ++        ) xs (concat xss)
```

- Ese hueco es el lugar de la lambda, y por eso el recuadro se ensancha

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Técnica extendida

- Nombrar los agujeros y volverlos parámetros

```
todosCincos  :: [Int] -> Bool  
todosCincos [] = True  
todosCincos (n : ns) = (\m b -> m == 5 && b) n (todosCincos ns)  
  
totalMonedas :: [Persona] -> Int  
totalMonedas [] = 0  
totalMonedas (p : ps) = (\p n -> cantMonedas p + n) p (totalMonedas ps)  
  
concat :: [[a]] -> [a]  
concat [] = []  
concat (xs : xss) = (\ys zs -> ys ++ zs) xs (concat xss)
```

- Los agujeros ya tienen nombre: el recuadro no menciona nada de afuera

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 2

- Recortar los recuadros y separarlos

```
todosCincos  :: [ ] ->   
todosCincos  []      =   
todosCincos  (n : ns) =  n (todosCincos ns)
```

```
totalMonedas :: [ ] ->   
totalMonedas []      =   
totalMonedas (p : ps) =  p (totalMonedas ps)
```

```
concat      :: [ ] ->   
concat      []      =   
concat      (xs : xss) =  xs (concat xss)
```

```
Int Bool  
True  
(\m b -> m == 5 && b)
```

```
Persona Int  
0  
(\p n -> cantMonedas p + n)
```

```
[a] [a]  
[]  
(\ys zs -> ys ++ zs)
```

■ Ahora sí: queda un **valor** y una **función de dos argumentos**

La técnica de los cuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 3: los tres esqueletos son el mismo

□ Escritas con los mismos nombres, son copias idénticas

<pre>elegirNombre :: [] -> [] elegirNombre [] = [] elegirNombre (x : xs) = [] x (elegirNombre xs)</pre>	<pre>Int Bool True (\m b -> m == 5 && b)</pre>
<pre>elegirNombre :: [] -> [] elegirNombre [] = [] elegirNombre (x : xs) = [] x (elegirNombre xs)</pre>	<pre>Persona Int 0 (\p n -> cantMonedas p + n)</pre>
<pre>elegirNombre :: [] -> [] elegirNombre [] = [] elegirNombre (x : xs) = [] x (elegirNombre xs)</pre>	<pre>[a] [a] [] (\ys zs -> ys ++ zs)</pre>

■ Tres copias del mismo esqueleto, y por eso alcanza con escribirlo una vez

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 3: los tres esqueletos son el mismo

- Hay que elegirle un nombre

```
elegirNombre ::                ->    -> [□] -> □  
elegirNombre □ □ []          = □  
elegirNombre □ □ (x : xs) = □ x (elegirNombre □ □ xs)
```

```
todosCincos  :: [Int] -> Bool  
todosCincos = elegirNombre (\m b -> m == 5 && b) True
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas = elegirNombre (\p n -> cantMonedas p + n) 0
```

```
concat      :: [[a]] -> [a]  
concat = elegirNombre (\ys zs -> ys ++ zs) []
```

- Abajo, cada función original es el esqueleto aplicado a **sus** recuadros

La técnica de los cuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 4

- Nombrar los huecos y las variables de tipo

```
elegirNombre :: (a -> b -> b) -> b -> [a] -> b  
elegirNombre f z [] = z  
elegirNombre f z (x : xs) = f x (elegirNombre f z xs)
```

```
todosCincos :: [Int] -> Bool  
todosCincos = elegirNombre (\m b -> m == 5 && b) True
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas = elegirNombre (\p n -> cantMonedas p + n) 0
```

```
concat :: [[a]] -> [a]  
concat = elegirNombre (\ys zs -> ys ++ zs) []
```

- Los huecos son los **parámetros**: el lugar que ocupaban ya estaba reservado

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Paso 5

□ Elegimos el nombre del esquema, en este caso **foldr**

```
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z [] = z  
foldr f z (x : xs) = f x (foldr f z xs)
```

```
todosCincos :: [Int] -> Bool  
todosCincos = foldr (\m b -> m == 5 && b) True
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas = foldr (\p n -> cantMonedas p + n) 0
```

```
concat :: [[a]] -> [a]  
concat = foldr (\ys zs -> ys ++ zs) []
```

■ **Plegado a derecha**: el esquema de la recursión estructural sobre listas

La técnica de los recuadros, otra vez

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

■ Se puede reescribir

- El recuadro puede llevar nombre

```
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z [] = z  
foldr f z (x : xs) = f x (foldr f z xs)
```

```
todosCincos :: [Int] -> Bool  
todosCincos = foldr combinar True      where combinar m b = m == 5 && b
```

```
totalMonedas :: [Persona] -> Int  
totalMonedas = foldr combinar 0      where combinar p n = cantMonedas p + n
```

```
concat      :: [[a]] -> [a]  
concat = foldr (++) []
```

- Es la misma definición con la lambda nombrada, y **(++)** ya tenía nombre

Lo mismo, manipulando la expresión

- El mismo esquema sale reescribiendo una sola función, con una regla que es la aplicación de lambda, leída al revés

□ $\dots \underline{x} \dots \underline{x} \dots = (\backslash y \rightarrow \dots y \dots y \dots) \underline{x}$ (sale $\underline{x}$)

- En `concat (xs : xss) = xs ++ concat xss`, se transforma así

$$\begin{aligned} & \text{xs ++ } \underline{\text{concat xss}} \\ = & (\backslash \text{zs} \rightarrow \underline{\text{xs}} \text{ ++ zs}) (\text{concat xss}) \\ = & (\backslash \text{ys zs} \rightarrow \text{ys ++ zs}) \text{xs} (\text{concat xss}) \end{aligned}$$

(sale el resultado recursivo)

(sale la cabeza, y azúcar sintáctico)

- Y eso ya es $f \text{ xs } (\text{concat xss})$, con $f = (\backslash \text{ys zs} \rightarrow \text{ys ++ zs})$

El tipo lo dice todo

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

- Antes de mirar qué hace, mirar qué tipo tiene

`foldr :: (a -> b -> b) -> b -> [a] -> b`

- ☐ Llamamos **b** al tipo del valor que describe el plegado
- ☐ El valor **z :: b** es el resultado para la lista vacía
- ☐ La función **f :: a -> b -> b** toma un elemento y el resultado del llamado recursivo con el resto

- Y las tres funciones de antes se escriben sin recursión explícita

- ☐ Lo que se gana no es el renglón, sino que la recursión la pone **foldr**

```
todosCincos = foldr (\m b -> m == 5 && b) True  
totalMonedas = foldr (\p n -> cantMonedas p + n) 0  
concat      = foldr (++) []
```

Comparar las dos versiones

■ Las dos versiones, una al lado de la otra

```
todosCincos :: [Int] -> Bool
todosCincos [] = True
todosCincos (n : ns) =
  n == 5 && todosCincos ns
```

```
totalMonedas :: [Persona] -> Int
totalMonedas [] = 0
totalMonedas (p : ps) =
  cantMonedas p + totalMonedas ps
```

```
todosCincos =
  foldr (\m b -> m == 5 && b)
    True
```

```
totalMonedas =
  foldr (\p n -> cantMonedas p + n)
    0
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x : xs) = f x (foldr f z xs)
```

Observar la inversión de los casos

- Lo marcado no desapareció: salió afuera y pasó a ser **argumento**
- Lo que era el esqueleto de cada una ahora está escrito **una sola vez**
- Y lo hace posible el **polimorfismo**: **foldr** no sabe de **Bool** ni de **Persona**

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

```
1  :  (2  :  (3  :  []))
```

- Plegar con `foldr f z` reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

```
1  :  (2  :  (3  :  []))
```

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

```
1  :  (2  :  (3  :  []))
```

- Plegar con `foldr f z` reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

```
1  \f\ (2  :  (3  :  []))
```

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

```
1  :  (2  :  (3  :  []))
```

- Plegar con `foldr f z` reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

```
1  'f' (2  'f' (3  :  []))
```


Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

```
1  :  (2  :  (3  :  []))
```

- Plegar con `foldr f z` reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

```
1  'f' (2  'f' (3  'f' []))
```

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

```
1  :  (2  :  (3  :  []))
```

- Plegar con `foldr f z` reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

```
1  'f' (2  'f' (3  'f' z))
```

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

`1 : (2 : (3 : []))`

- Plegar con **`foldr f z`** reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

`1 'f' (2 'f' (3 'f' z)) = f 1 (f 2 (f 3 z))`

- Por eso **`foldr (:) []`** es la identidad

☐ Reemplaza cada constructor por sí mismo

- ¿Cómo se llama este esquema en Smalltalk o en Java?

Qué es plegar, en realidad

- Una lista **queda definida** por sus constructores: `[]` y `(:)`

`1 : (2 : (3 : []))`

- Plegar con **`foldr f z`** reemplaza cada constructor: `(:)` por `f`, `[]` por `z`

`1 'f' (2 'f' (3 'f' z)) = f 1 (f 2 (f 3 z))`

- Por eso **`foldr (:) []`** es la identidad

- ☐ Reemplaza cada constructor por sí mismo

- ¿Cómo se llama este esquema en Smalltalk o en Java?

- ☐ En Smalltalk, **`inject:into:`**

- ☐ En Java, **`reduce`** de los **`Stream`**

- ☐ Los dos pliegan para el otro lado

Currificado, fabrica funciones de listas

- Leamos el tipo con los paréntesis puestos

`foldr :: (a -> b -> b) -> b -> ([a] -> b)`

- Leído así, **foldr** toma una **operación** y un **valor** y describe una **función de listas**

<code>(a -> b -> b), b</code>	$\mapsto$	<code>[a] -> b</code>
<code>(&&), True</code>	$\mapsto$	<code>and</code>
<code>(++) , []</code>	$\mapsto$	<code>concat</code>
<code>(:), []</code>	$\mapsto$	<code>id</code>

- Por eso las instancias se pueden escribir **sin nombrar la lista**

map y filter eran casos de esto

- Redefinimos los esquemas anteriores con **foldr**
- ¿Con qué **argumentos**?

```
map f      = foldr    
filter p = foldr  
```

```
map f []      = []  
map f (x : xs) = f x : map f xs  
  
filter p []      = []  
filter p (x : xs) =  
    if p x  
    then x : filter p xs  
    else filter p xs  
  
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z []      = z  
foldr f z (x : xs) = f x (foldr f z xs)
```

map y filter eran casos de esto

- Redefinimos los esquemas anteriores con **foldr**
- ¿Con qué **argumentos**?

```
map f      = foldr (\x ys -> f x : ys) []  
filter p = foldr  
```

```
map f []      = []  
map f (x : xs) = f x : map f xs  
  
filter p []      = []  
filter p (x : xs) =  
    if p x  
    then x : filter p xs  
    else filter p xs  
  
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z []      = z  
foldr f z (x : xs) = f x (foldr f z xs)
```

map y filter eran casos de esto

```
map f []           = []
map f (x : xs)     = f x : map f xs

filter p []        = []
filter p (x : xs) =
  if p x
  then x : filter p xs
  else filter p xs

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []       = z
foldr f z (x : xs) = f x (foldr f z xs)
```

- Redefinimos los esquemas anteriores con **foldr**
- ¿Con qué **argumentos**?

```
map f      = foldr (\x ys -> f x : ys) []
filter p   = foldr (\x ys -> if p x then x : ys else ys) []
```

- Los dos esquemas de la clase pasada son **instancias** del plegado
- Era la respuesta a la pregunta del principio: sí, hay uno más general



Recursión estructural

Tres definiciones conocidas

- ¿Hasta dónde llega **foldr**?
- Tres funciones sobre listas que ya vimos escritas a mano

```
suma :: [Int] -> Int
suma []      = 0
suma (x : xs) = x + suma xs
```

```
isort :: Ord a => [a] -> [a]
isort []      = []
isort (x : xs) = insertarOrdenado x (isort xs)
```

```
(++) :: [a] -> [a] -> [a]
[]      ++ ys = ys
(x : xs) ++ ys = x : (xs ++ ys)
```

- ¿Se pueden escribir con **foldr**?

Sí, las tres son plegados

■ Las tres se escriben con **foldr**

- ☐ sin recursión explícita

```
suma = foldr    
isort = foldr    
(++) = foldr  
```

```
suma []           = 0  
suma (x : xs)    = x + suma xs  
  
isort []          = []  
isort (x : xs)    = insertarOrdenado x (isort xs)  
  
[] ++ ys          = ys  
(x : xs) ++ ys   = x : (xs ++ ys)  
  
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z []      = z  
foldr f z (x : xs) = f x (foldr f z xs)
```

Sí, las tres son plegados

■ Las tres se escriben con **foldr**

- ☐ sin recursión explícita

```
suma = foldr (+) 0
```

```
isort = foldr  
```

```
(++) = foldr  
```

```
suma []           = 0
suma (x : xs)     = x + suma xs

isort []          = []
isort (x : xs)    = insertarOrdenado x (isort xs)

[] ++ ys          = ys
(x : xs) ++ ys    = x : (xs ++ ys)

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x : xs) = f x (foldr f z xs)
```

Sí, las tres son plegados

■ Las tres se escriben con **foldr**

- ☐ sin recursión explícita

```
suma = foldr (+) 0
isort = foldr insertarOrdenado []
(++ ) = foldr  
```

```
suma []           = 0
suma (x : xs)     = x + suma xs

isort []          = []
isort (x : xs)    = insertarOrdenado x (isort xs)

[] ++ ys          = ys
(x : xs) ++ ys    = x : (xs ++ ys)

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x : xs) = f x (foldr f z xs)
```

Sí, las tres son plegados

■ Las tres se escriben con **foldr**

- sin recursión explícita

```
suma = foldr (+) 0
isort = foldr insertarOrdenado []
(++ ) = foldr (\x h ys -> x : h ys) id
```

```
suma []           = 0
suma (x : xs)     = x + suma xs

isort []          = []
isort (x : xs)    = insertarOrdenado x (isort xs)

[] ++ ys          = ys
(x : xs) ++ ys    = x :  $\underbrace{(xs ++ ys)}_h$ 

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x : xs) = f x (foldr f z xs)
```

■ En **(++)** el plegado no describe una lista: describe una **función**

- Acá **b** es **[a] -> [a]**, y por eso el caso base es **id**
- Y **h**, el resultado del llamado recursivo con el resto, no se combina: se **aplica**

■ ¿Qué funciones, exactamente, se pueden escribir con **foldr**?

La forma que comparten

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []           = z
foldr f z (x : xs) = f x (foldr f z xs)
```

- Todas se definen por **dos ecuaciones**

- Una por constructor de lista

- Y cada ecuación tiene restringido **lo que puede usar**

- ☐ El caso base no depende de nada
- ☐ El caso recursivo usa x y el resultado de la llamada, y nada más

$$\begin{aligned} g [] &= \dots \\ g (x : xs) &= \dots \times \dots (g xs) \dots \end{aligned}$$

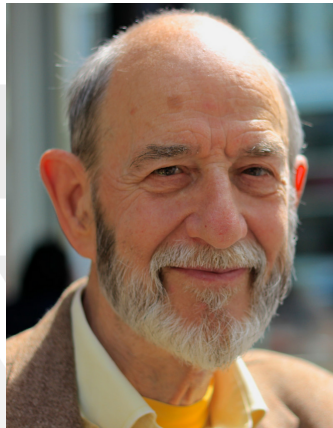
- A eso lo llamamos **recursión estructural**

- ☐ El nombre viene de un artículo de **Rod Burstall**, de 1969

- Y una definición con esa forma **es equivalente** a una aplicación de **foldr**

Rod Burstall

Fue un científico de la computación británico, uno de los cuatro fundadores del Laboratorio de Fundamentos de Ciencias de la Computación (LFCS) de la Universidad de Edimburgo, junto con Robin Milner. En 1969 publicó *Proving properties of programs by structural induction*, el artículo que puso en el centro la idea que organiza esta clase: definir las funciones siguiendo la forma del tipo de datos, y demostrar sus propiedades por inducción sobre esa misma forma. Diseñó los lenguajes POP-2 y Hope, donde aparecieron los tipos algebraicos con *pattern matching* que después heredaron ML, Miranda y Haskell. También trabajó en transformación de programas: reescribir una definición en otra equivalente, pero mejor.



11 de noviembre de 1934 –
13 de febrero de 2025

Las dos restricciones, con precisión

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x : xs) = f x (foldr f z xs)
```

1. El caso base es un valor fijo
 - ☐ No depende de g , que no se puede usar en esta parte
 2. El caso recursivo **no puede usar** g ni xs sueltas
 - ☐ Solo puede usar x y la expresión $(g\ xs)$
- La segunda es la que hace la mayoría del trabajo
 - Prohíbe mirar la cola de la lista, y prohíbe llamarse con otra cosa
 - Es justo lo que **foldr** le pasa a **f**:
el elemento y el resultado del llamado recursivo con el resto

Dos que sí entran, aunque no lo parezcan

- Es estructural aunque **no use la cabeza**

```
length :: [a] -> Int
length []      = 0
length (_ : xs) = 1 + length xs
```

- Y es estructural aunque **no use el resultado del llamado recursivo con el resto**

```
head :: [a] -> a
head []      = error "no tiene cabeza"
head (x : _) = x
```

- El esquema pide que no use **nada más**, no que use todo

Qué queda afuera

- Una definición que **no** cumple la segunda restricción

```
-- NO es estructural: usa xs suelta, no solo (g xs)
insertarOrdenado :: Ord a => a -> [a] -> [a]
insertarOrdenado y [] = [y]
insertarOrdenado y (x : xs) =
  if y <= x
  then y : x : xs
  else x : insertarOrdenado y xs
```

- En la rama del **then** aparece **xs** sin recursión
- Es una definición perfectamente válida
 - ☐ Simplemente **no entra en el esquema**
 - ☐ Y tal como está escrita, no es una aplicación de **foldr**

Y otro, que queda más afuera

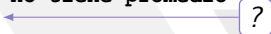
- Otra que no es estructural, y por un motivo distinto

```
-- Selection sort
ssort :: Ord a => [a] -> [a]
ssort []      = []
ssort (x : xs) = mínimo (x : xs)
                  : ssort (sacarMínimo (x : xs))
```

- La llamada recursiva no es sobre **xs**: es sobre **otra lista**
- Termina igual, porque esa lista es más corta
 - Pero eso ya no lo garantiza la **forma** de la definición
- ¿Qué esquema es, entonces?

Y uno donde un solo resultado no alcanza

- Y uno más, donde el problema es anterior al esquema

```
promedio :: [Float] -> Float
promedio [] = error "no tiene promedio"
promedio (x : xs) =  ← 
```

- Con **promedio xs** no alcanza para completar la segunda ecuación

```
promedio [3] → 3      promedio [2,4] → 3
promedio (0 : [3]) → 1.5  promedio (0 : [2,4]) → 2
```

- Los dos promedios de arriba son el mismo, y los de abajo no
- El promedio del resto no dice sobre cuántos se promedió

```
promedio xs = suma xs / longitud xs
  where suma      = foldr (+) 0
        longitud = foldr (\_ n -> n + 1) 0
```

- Sale, pero la recursión no está en **promedio**: está en los plegados

Entonces, ¿por qué aceptar una restricción?

- Porque la restricción **compra algo**
 - ☐ Igual que en la clase pasada
- Un esquema es un constructo explícito
 - ☐ Y un constructo explícito admite una regla de razonamiento
- Sobre la recursión estructural vale la **inducción estructural**
 - ☐ Es el tema de la clase que viene
- Sobre una recursión general no se puede decir casi nada

Una propiedad del esquema

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

- El esquema no solo admite inducción
 - También cumple propiedades **por ser un plegado**

Para todas **f** y **g**, y todo **z**:

foldr (f . g) z = foldr f z . map g

- En **totalMonedas**, con **f** como **(+)** y **g** como **cantMonedas**

```
totalMonedas = foldr (\p n -> cantMonedas p + n) 0  
              = foldr ((+) . cantMonedas) 0  
              = foldr (+) 0 . map cantMonedas
```
- Se demuestra **una sola vez**, y vale para todas las **f**, **g** y **z**

Para qué sirve darla vuelta

```
data Persona = ...  
cantMonedas :: Persona -> Int  
-- función costosa
```

- De las dos formas, la de la derecha **aísla el trabajo caro**

```
totalMonedas = foldr (+) 0 . map cantMonedas
```

- El **map** se puede repartir entre muchas máquinas

- ☐ Cada una hace su parte de los **cantMonedas**, que es lo que cuesta

- Es la forma de **MapReduce**, con la que Google indexó la web

- Para repartir también el plegado hace falta que **f** sea **asociativa**

- ☐ Sobre eso vamos a volver

- Derivar programas aplicando leyes así tiene nombre propio

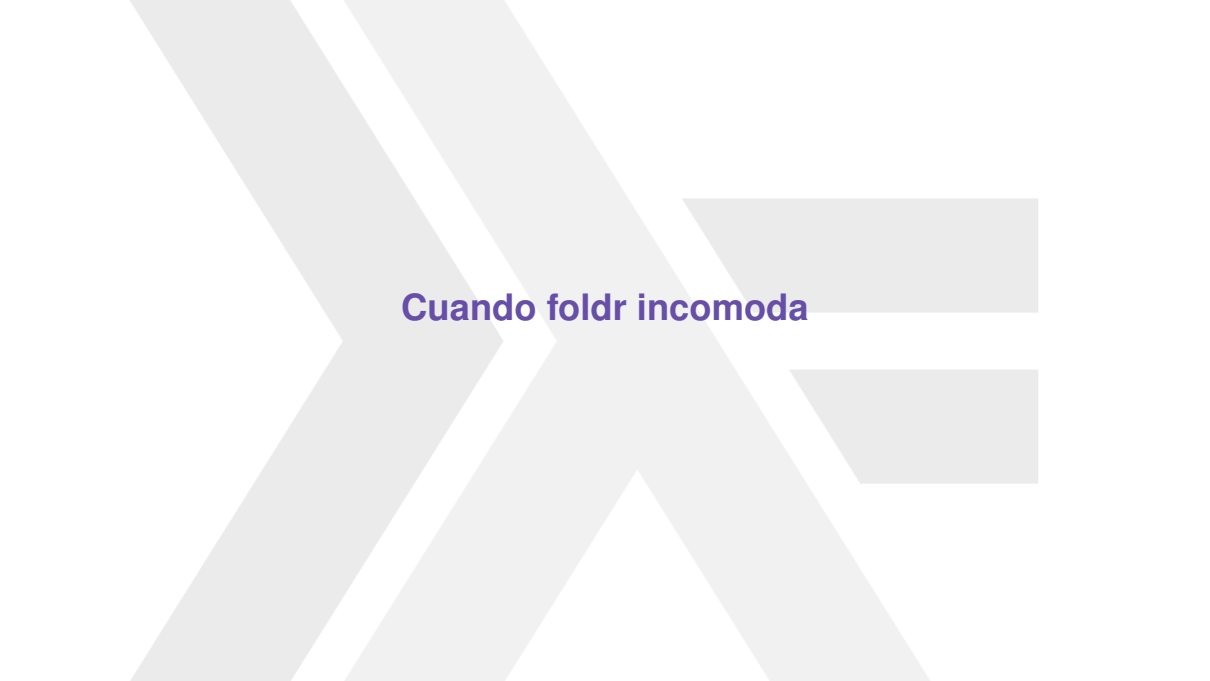
- ☐ El **formalismo Bird-Meertens**

Richard Bird

Fue un científico de la computación inglés, profesor en la Universidad de Oxford y director de su Computing Laboratory. Con Lambert Meertens desarrolló lo que hoy se llama el **formalismo Bird-Meertens**: un cálculo para derivar programas a partir de su especificación aplicando leyes algebraicas, como la que acabamos de ver. La idea de fondo es que un programa no se escribe primero y se verifica después, sino que se *calcula*. Fundó la columna *Functional Pearls* del Journal of Functional Programming y escribió varios de los libros con los que se enseña programación funcional. *Introduction to Functional Programming* (1988), el "Bird-Wadler", salió de su colaboración con Phil Wadler en el Programming Research Group de Oxford, el grupo que por entonces dirigía Tony Hoare y donde en esos mismos años estaba también John Hughes. Wadler y Hughes iban a integrar, poco después, el comité que diseñó Haskell. *The Algebra of Programming* (1997) lo escribió con Oege de Moor.



13 de febrero de 1943 –
4 de abril de 2022



Cuando foldr incomoda

Un caso que no entra

■ Volvamos a **insertarOrdenado**

- En una rama necesita **xs**
- La cola original, no el resultado de la recursión

■ Pero **foldr** no se la da: solo pasa (*g xs*)

■ Necesitamos un esquema que dé **las dos cosas**

```
recr :: b -> (a -> [a] -> b -> b) -> [a] -> b
recr z f [] = z
recr z f (x : xs) = f x xs (recr z f xs)
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x : xs) = f x (foldr f z xs)

insertarOrdenado y [] = [y]
insertarOrdenado y (x : xs) =
  if y <= x
  then y : x : xs
  else x : insertarOrdenado y xs
```

Al revés que en **foldr**: el caso base va primero
Es a propósito, para no confundir los esquemas

■ **Recursión primitiva**: la diferencia con **foldr** es ese **xs** de más

■ **Ojo**: tener **xs** a mano invita a recorrerla de nuevo

- Si en cada paso se llama a otra función recursiva sobre **xs**,
el costo pasa a ser **cuadrático**

Ahora sí entra

- El esquema **recr** le pasa la cola que faltaba
 - Y la definición queda sin recursión explícita

insertarOrdenado y =

recr

```
recr :: b -> (a -> [a] -> b -> b) -> [a] -> b
recr z f [] = z
recr z f (x : xs) = f x xs (recr z f xs)

insertarOrdenado y [] = [y]
insertarOrdenado y (x : xs) =
  if y <= x
  then y : x : xs
  else x : insertarOrdenado y xs
```

Ahora sí entra

- El esquema **recr** le pasa la cola que faltaba

- ☐ Y la definición queda sin recursión explícita

```
insertarOrdenado y =  
  recr [y] (\x xs rec ->  
    if y <= x  
    then y : x : xs  
    else x : rec)
```

```
recr :: b -> (a -> [a] -> b -> b) -> [a] -> b  
recr z f [] = z  
recr z f (x : xs) = f x xs (recr z f xs)  
  
insertarOrdenado y [] = [y]  
insertarOrdenado y (x : xs) =  
  if y <= x  
  then y : x : xs  
  else x : insertarOrdenado y xs
```

- Toda función estructural es primitiva (ignorando el **xs**)

- El esquema **recr** parece más expresivo, pero no lo es

- ☐ Se lo puede escribir con **foldr** más algún ajuste
- ☐ El esquema es **recr z f = ajuste (foldr [] [])**
 - ☐ donde el **foldr** no pliega a **b**, y
 - ☐ la auxiliar **ajuste** puede ser **(\f x -> f x x)**, **fst**, **snd**, ...
- ☐ Lo que cambia es la comodidad, no el poder



Plegar al revés

Un inconveniente de foldr

- En **foldr**, la llamada recursiva es **argumento** de otra función

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []      = z
foldr f z (x : xs) = f x (foldr f z xs)
```

argumento de f

- Si **f** necesita su segundo argumento, primero hay que llegar al final

- ☐ Y recién ahí empieza a combinar

```
foldr (+) 0 [1,2,3]
→ 1 + (foldr (+) 0 [2,3])
→ 1 + (2 + (foldr (+) 0 [3]))
→ 1 + (2 + (3 + 0))
```

- ☐ Ninguna suma se hace hasta el final, y mientras tanto la expresión crece

- ¿Se podrá dar vuelta, y que la llamada recursiva sea **la función principal**?

¿Por qué queríamos eso?

- Porque entonces **la recursión determina qué se computa**

- ☐ Y no queda envuelta en otra llamada

- Porque puede mejorar la eficiencia

- ☐ No hay que apilar el trabajo pendiente

- A una recursión así se la llama **recursión a la cola**

- ☐ La llamada recursiva es lo último y único que se hace

Plegado a izquierda

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []          = z
foldr f z (x : xs) = f x (foldr f z xs)
```

- El esquema que sale de dar vuelta la recursión

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []          = z
foldl f z (x : xs) = foldl f (f z x) xs
```

- El resultado parcial viaja **hacia adentro** como acumulador
- La lista se recorre desde el principio

```
foldl (+) 0 [1,2,3]
→ foldl (+) (0+1) [2,3]
→ foldl (+) ((0+1)+2) [3]
→ foldl (+) (((0+1)+2)+3) []
→ ((0+1)+2)+3
```

Repetimos la de **foldr**, para contrastar

```
foldr (+) 0 [1,2,3]
→ 1 + (foldr (+) 0 [2,3])
→ 1 + (2 + (foldr (+) 0 [3]))
→ 1 + (2 + (3 + 0))
```

Pero en Haskell tampoco se apila menos

- Mirando de nuevo la expansión de recién, el acumulador **no se evalúa**

```
foldl (+) 0 [1,2,3]
→ foldl (+) (0+1) [2,3]
→ foldl (+) ((0+1)+2) [3]
```

- ☐ Así que **foldl** también apila, igual que **foldr**

- La diferencia es que el de **foldl** se puede arreglar

- ☐ Forzando el acumulador en cada paso, o sea haciéndolo **estricto**
- ☐ Eso es **foldl'**, que Haskell trae en **Data.List**

- El de **foldr** no: ahí lo pendiente es la **forma** de la expresión

- ☐ No hay un acumulador donde forzar

Un **foldl** es una repetición

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []          = z
foldl f z (x : xs) = foldl f (f z x) xs
```

■ El esquema **foldl** es un **operador de iteración**

La misma repetición, en imperativo

```
función foldl f z xs {
  ac      := z
  resto := xs
  mientras resto no es vacía {
    ac      := f ac (head resto)
    resto := tail resto
  }
  devolver ac
}
```

- ☐ La variable **ac** arrastra el resultado parcial, y es el acumulador del **foldl**
- ☐ La variable **resto** se achica en cada vuelta, y hace terminar el ciclo
 - ☐ Y no todo **while** es un **foldl**: **while (true) {}** no termina
- ☐ En el ciclo no se apila nada, porque es estricto por construcción

Mejor dicho, un **foreach**

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []          = z
foldl f z (x : xs) = foldl f (f z x) xs
```

■ El **while** de la diapo anterior muestra el **mecanismo**

- ☐ Hay que escribir el resto, achicarlo, y preguntar si se acabó
- ☐ Y por eso va primero: es lo que deja ver por qué el ciclo termina

La misma repetición, expresada como repetición acotada

```
función foldl f z xs {
  ac := z
  for (const x of xs) {
    ac := f ac x
  }
  devolver ac
}
```

- ☐ Todo **foldl** es un **foreach** con acumulador
- ☐ La correspondencia es más exacta que con el **while**, pero no es total
 - ☐ Al **while** hay que pedirle que **termine**
 - ☐ Al **foreach**, que el cuerpo **sea sólo el acumulador**

Las dos formas, lado a lado

- Los dos plegados, desplegados sobre una lista cualquiera

```
foldl f z [x1, x2, ..., xn] = (...((z `f` x1) `f` x2) `f` ... `f` xn)
```

```
foldr f z [x1, x2, ..., xn] = (x1 `f` (x2 `f` ... `f` (xn `f` z) ...))
```

- Asocian para lados opuestos, y el **z** queda en puntas opuestas
- El acumulador y el resultado del llamado recursivo con el resto van a lados opuestos
 - ☐ En **foldl**, el acumulador va **a la izquierda** de **f**
 - ☐ En **foldr**, el resultado del llamado recursivo con el resto va **a la derecha**
- ¿Cuándo dan lo mismo?

Primer teorema de dualidad

Si **f** es **asociativa** y **z** es su **neutro**, entonces para toda lista **finita xs**:

$$\text{foldr } f \ z \ xs = \text{foldl } f \ z \ xs$$

- Por eso **sum**, **product** y **concat** dan igual con los dos
- Y por eso con la resta **no** dan igual
 - La resta no es asociativa

Pero coinciden sin ser la misma f

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z (x : xs) = f x (foldr f z xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z (x : xs) = foldl f (f z x) xs
```

- Dos formas de dar vuelta una lista, una con cada plegado

```
reverse = foldr postfix []
rev      = foldl prefix  []
```

```
postfix x xs = xs ++ [x]
prefix  xs x = [x] ++ xs
```

- Ninguna de las dos es asociativa, y sin embargo **reverse = rev**
- El primer teorema no alcanza: hace falta uno más general

Segundo teorema de dualidad

Si para todos x, y, z valen

$$x \text{ `f` } (y \text{ `g` } z) = (x \text{ `f` } y) \text{ `g` } z$$

$$x \text{ `f` } z0 = z0 \text{ `g` } x$$

entonces, para toda lista finita xs :

$$\text{foldr } f \text{ } z0 \text{ } xs = \text{foldl } g \text{ } z0 \text{ } xs$$

- El primer teorema es el caso particular $f = g$
- Y $\text{reverse} = \text{rev}$ sale de acá, tomando f como postfix y g como prefix

Tercer teorema de dualidad

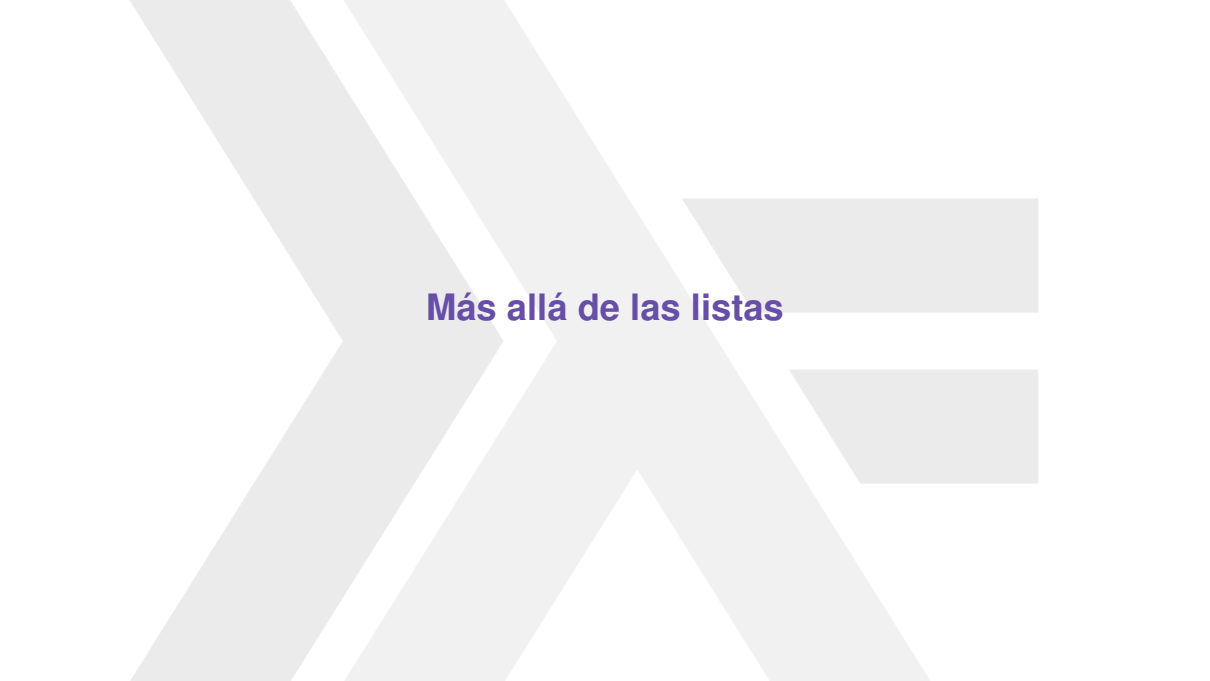
Para toda lista finita **xs**:

```
foldr f z xs = foldl (flip f) z (reverse xs)
```

- Sin hipótesis sobre **f**: siempre vale
- Dice que los dos plegados son **el mismo**, mirado desde la otra punta

Una diferencia que ningún teorema arregla

- Los tres teoremas piden lista **finita**, y no es un detalle
- El esquema **foldl** no puede dar ningún resultado hasta llegar al final
 - Si la lista es infinita, no termina nunca
- El de **foldr** sí puede, si **f** no necesita su segundo argumento
`foldr (\x r -> True) False [1..] → ... → True`
- Es la evaluación no estricta de la clase 1, otra vez



Más allá de las listas

Los tipos que ya veníamos usando

- Algunos tipos vienen con el lenguaje

`Bool` `Char` `Int` `Float` `(a, b)` `(a -> b)` `[a]`

- Y otros los declaramos nosotros, ya desde la clase 0

- Días de la semana, valores opcionales, árboles

- Todos los declarados salen de la misma cláusula

`data Tipo =` *declaración de los constructores*

Un tipo se declara dando sus constructores

```
data Día      = Lun | Mar | Mié | Jue | Vie | Sáb | Dom
```

```
data Persona = LaPersona String String Int
```

```
data Maybe a = Nothing | Just a
```

- Cada constructor dice qué partes tiene un valor
- Y la declaración dice que **esos son todos**: no hay otros
- Se llaman **algebraicos** porque combinan suma y producto
 - Suma: hay varios constructores entre los que elegir, como en **Día**
 - Producto: un constructor lleva varios campos, como **LaPersona**
 - El tipo **Maybe** tiene las dos cosas a la vez

Los constructores son funciones

- Declarar un tipo es declarar **también** estas funciones

```
Lun      :: Día
LaPersona :: String -> String -> Int -> Persona
Just     :: a -> Maybe a
Bin      :: AB a -> a -> AB a -> AB a
```

- ☐ Un constructor sin campos ya es un valor de **orden 0**
- ☐ Con campos, es una función que arma uno

- Pero son las únicas funciones **sin ecuaciones**

- ☐ Sin ecuación no hay nada que reducir
- ☐ Y si queda algo pendiente es en los argumentos, como en **Just (2+3)**
 - ☐ Y ese **2+3** no se reduce hasta que haga falta
- ☐ Es el primer caso de “¿Cuándo para la reducción?” de la clase 1

Y también sirven para preguntar

- Un constructor también vale del lado **izquierdo** del =
`apellido (LaPersona _ ap _) = ap`
- Eso es **pattern matching**
 - Pregunta con qué constructor se armó el valor
 - Y de paso les pone nombre a los campos
- Se puede justamente porque no tienen ecuaciones
 - Un constructor aplicado no reduce, así que queda a la vista
- Es lo que venimos haciendo toda la clase
 - `foldr f z (x : xs)` pregunta si la lista se armó con `(:)`

El caso general

```
data Nat          = Zero | Succ Nat
data RoseTree a  = Rose a [RoseTree a]
data Red          = Fin | Filtro (Int -> Bool) Red
```

- Un constructor lleva los campos que quiera, de los tipos que quiera
- Un campo es **recursivo** cuando el tipo que se declara aparece en su tipo
 - Y puede aparecer **adentro** de otro
 - En **Rose** el campo es de tipo **[RoseTree a]**
- Los valores se construyen aplicando constructores
 - Leyendo la declaración de forma **inductiva**: cada valor, con **finitas** aplicaciones

La lectura inductiva no es todo el tipo

- En Haskell el tipo tiene además valores **infinitos**
 - Se escribe `infinito = Succ infinito`, y es un `Nat`
 - Como el `[1..]` de la clase 1
- Ningún cómputo aplica infinitos constructores: aplica finitos, pero **sin cota**
- En la denotación sí están: son el **límite** de esas aproximaciones

Y cómo se usa un valor de ese tipo

- Los constructores dicen cómo se **arma** un valor
- Falta decir cómo se **usa**: una ecuación por constructor

```
maybe z f Nothing = z  
maybe z f (Just x) = f x
```

- Y el tipo del esquema sale solo de esas dos ecuaciones
- ```
maybe :: b -> (a -> b) -> Maybe a -> b
```

- Un parámetro por constructor, cada uno recibiendo sus campos

## Ese esquema tiene dos nombres

- Es el **análisis por casos** de **Maybe**
  - Lo que en Haskell se escribe con **case**
- Y también es su **plegado**
  - Sin campos recursivos, los dos son lo mismo
- Todo lo que se hace con un **Maybe** sale de ahí

```
fromMaybe d = maybe d id
isJust = maybe False (const True)
aplicar f = maybe Nothing (Just . f)
```

## Las listas y los árboles, otra vez

```
-- [a] ya viene predefinido; si se declarara, sería así:
-- data [a] = [] | a : [a]
```

```
data AB a = Nil | Bin (AB a) a (AB a)
```

- En `[a]`, `[]` es el constructor base y `(:)` el recursivo
  - Y `foldr` es el observador que sale de esa declaración
- En `AB a`, el constructor `Nil` es base, y `Bin` lleva **dos** campos recursivos
- O sea que toda la clase venía hablando de esto sin decirlo

## La receta

- **Regular**: el campo recursivo es **exactamente** del tipo que se declara
- El esquema de un tipo regular sale de sus constructores
  1. Un parámetro por constructor
  2. Cada uno recibe los campos de su constructor
  3. En los campos **recursivos** llega el plegado del campo, no el campo
- Para listas (que son regulares): dos constructores, dos parámetros, **z** y **f**
- El tipo **RoseTree** no es regular
  - El campo de **Rose** es de tipo **[RoseTree a]**, no **RoseTree a**
  - Para su esquema hay más de una convención, y lo vemos más adelante

## Aplicando la receta a los árboles

- Los constructores del tipo, que son de donde sale todo

```
data AB a = Nil | Bin (AB a) a (AB a)
```

- El constructor **Nil** no lleva campos: un valor de orden 0

- El constructor **Bin** lleva tres campos, y **dos son recursivos**

```
foldAB :: b -> (b -> a -> b -> b) -> AB a -> b
```

```
foldAB z g Nil = z
```

```
foldAB z g (Bin ti x td) = g (foldAB z g ti) x (foldAB z g td)
```

- ☐ Y llegan los resultados de plegar los subárboles, no los subárboles

## Y funciona igual

```
data AB a = Nil | Bin (AB a) a (AB a)
foldAB :: b -> (b -> a -> b -> b) -> AB a -> b
foldAB z g Nil = z
foldAB z g (Bin ti x td) = g (foldAB z g ti) x (foldAB z g td)
```

- Tres funciones sobre árboles, ninguna con recursión explícita

```
cantNodos = foldAB 0 (\n x m -> n + 1 + m)
espejo = foldAB Nil (\ti x td -> Bin td x ti)
inorder = foldAB [] (\xs x ys -> xs ++ [x] ++ ys)
```

- Ninguna menciona la recursión: la puso el esquema
- El programador solo dice **qué hacer en cada constructor**

# Un tipo que no significa nada

- Un tipo recursivo cualquiera, sin ninguna intención de significado

```
data TG = CB
 | CC TG Char TG
 | CD Int TG
 | CE TG TG TG
 | CF TG Char
```

- ☐ ¿Cuántos parámetros va a tener su esquema?
- ☐ ¿De qué tipos son?
- ☐ ¿Hace falta entender qué representa el tipo para contestar?



## Y el esquema sale igual

- Un parámetro por constructor, con los campos de cada uno

```
foldTG :: b
 -> (b -> Char -> b -> b)
 -> (Int -> b -> b)
 -> (b -> b -> b -> b)
 -> (b -> Char -> b)
 -> TG -> b
```

- ☐ Y los campos **TG** llegan plegados, o sea que son **b**
- ☐ Así, la implementación del código también es mecánica

- Y con eso ya se puede definir, sin saber qué es un **TG**

```
cantCharTG :: TG -> Int
cantCharTG = foldTG 0
 (\n1 ch n2 -> n1 + 1 + n2)
 (\m n -> n)
 (\n1 n2 n3 -> n1 + n2 + n3)
 (\n ch -> n + 1)
```

El **m** **NO** es resultado del plegado

## El aviso de **recr**, con un ejemplo

- La función **ancestrosDe** da los ancestros de un elemento del árbol, de tipo **Eq a => AB a -> a -> [a]**
- Es **parcial**: si el elemento no está, no hay respuesta que dar
  - Y **[]** no sirve para decirlo: **[]** significa que está, y es la raíz
- La solución directa pregunta **estáEn y ti** para saber por dónde bajar
  - Pero **estáEn** recorre el subárbol **entero**, y eso pasa en cada nodo
  - Algo que se resuelve en **una** pasada termina costando **cuadrático**

```
ancestrosDe :: Eq a => AB a -> a -> [a]
ancestrosDe Nil y = []
ancestrosDe (Bin ti x td) y =
 if x == y then []
 else if estáEn y ti
 then x : ancestrosDe ti y
 else x : ancestrosDe td y
```

Orden lineal sobre **ti**, y en cada nodo

## La salida: total adentro, parcial afuera

- La auxiliar describe un **Maybe**, y así **sí** entra en el esquema

```
ancestrosDe t y = fromJust (buscarAncestros t y)
```

```
buscarAncestros :: Eq a => AB a -> a -> Maybe [a]
```

- Describe **Nothing** si no está, y **Just []** si está y es la raíz
- Cada subárbol llega con **su** respuesta, así que alcanza una pasada
  - Ya no hace falta preguntar dónde está: el plegado lo trae
- Y la parcialidad quedó en la **proyección**, no en la recursión

```
buscarAncestros t y = foldAB Nothing combinar t
 where combinar mxs x mys =
 if x == y
 then Just []
 else case (mxs, mys) of
 (Just xs, _) -> Just (x : xs)
 (_, Just xs) -> Just (x : xs)
 _ -> Nothing
```

## Un tipo que se apoya en otro

- El campo recursivo de **Rose** es una **lista** de **RoseTree**

- ☐ Así que recorrer el árbol pasa por recorrer una lista

```
data RoseTree a = Rose a [RoseTree a]
```

```
sumRose :: RoseTree Int -> Int
```

```
sumRose (Rose x ts) = x + sum (map sumRose ts)
```

- ☐ El **map** aplica la recursión a **todos**, y el **sum** combina los resultados
- ☐ O sea que la recursión sobre el árbol se apoya en dos esquemas sobre **listas**

## Sin orden superior, aparece la recursión mutua

- Sin **map** ni **sum**, el recorrido de la lista se escribe aparte

- ☐ Una **subtarea**, que trabaja sobre la lista de hijos

```
sumRose' (Rose x ts) = x + aRaTyS ts
```

```
aRaTyS :: [RoseTree Int] -> Int -- Aplicar Recursión a Todos y Sumar
```

```
aRaTyS [] = 0
```

```
aRaTyS (t : ts) = sumRose' t + aRaTyS ts
```

- ☐ Las dos se llaman entre sí: es **recursión mutua**
- ☐ Y cada una es estructural sobre **su** tipo, el árbol y la lista
- ☐ La salida **fea** es partir la lista adentro del patrón del árbol
  - ☐ Mezcla las dos tareas en una, y ninguna queda estructural

## Y por eso hay más de una convención

- Espejar un **RoseTree** es dar vuelta la lista de hijos, en cada nodo

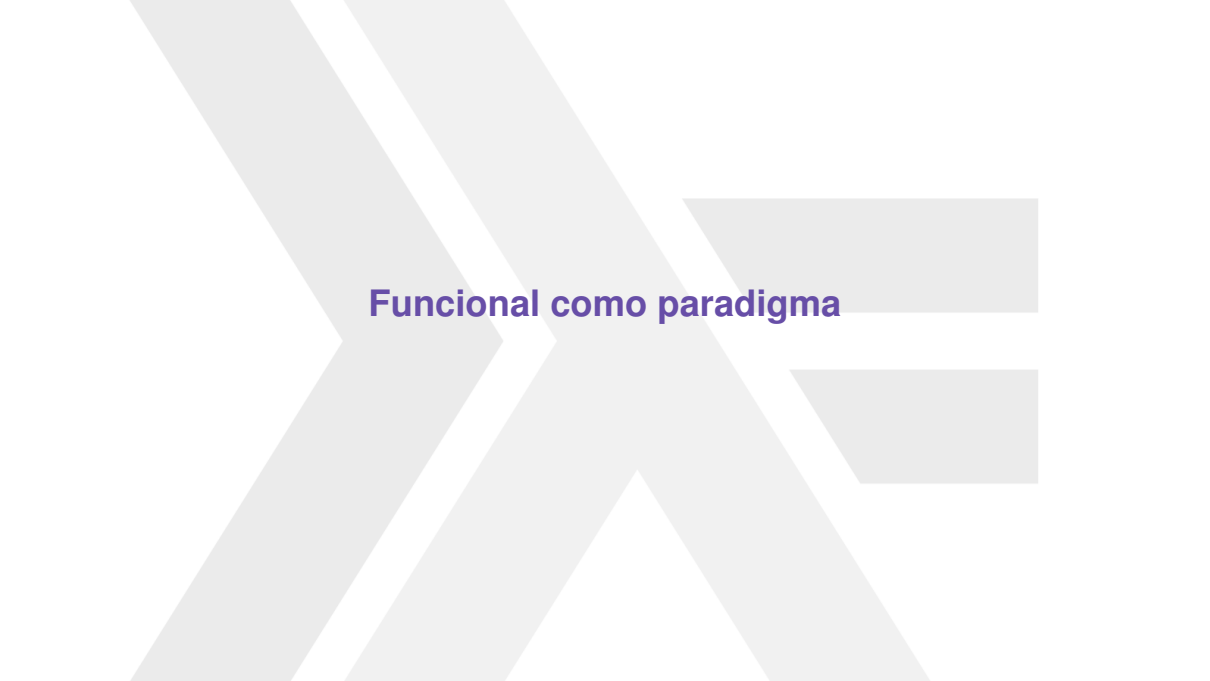
```
mirrorRose :: RoseTree a -> RoseTree a
mirrorRose (Rose x ts) = Rose x (reverse (map mirrorRose ts))
```

- ☐ Es la expansión de un esquema que rearma el nodo con **Rose** y da vuelta la lista con **reverse**

- Aplicar la receta de la diapo 54 tal cual da este esquema

```
foldRose :: (a -> [b] -> b) -> RoseTree a -> b
foldRose h (Rose x ts) = h x (map (foldRose h) ts)
```

- ☐ Pero **no** es recursión estructural: **h** recibe la lista de resultados **entera**
  - ☐ La lista es parte del tipo, y su recorrido queda sin disciplina
- ☐ Para que lo sea hay que plegar también la lista, y ahí hay que elegir **cuánto** de ese recorrido se delega
- ☐ Cada elección da una convención distinta, y por eso no hay una sola



**Funcional como paradigma**

## Dónde encaja esto

- En la clase 1 dijimos que un paradigma tiene dos partes
  - ☐ Un **núcleo**
  - ☐ Y un **disciplinamiento** que se le impone encima
- Los esquemas de recursión son **una de las formas** de ese disciplinamiento
  - ☐ Otra son los **tipos**

### **Núcleo**

la recursión general, sin restricciones

### **Disciplinamiento**

la recursión estructural, y su esquema: el plegado  
el sistema de tipos



## Ojo: acá sí se pierde poder

- El disciplinamiento no siempre cuesta poder de cómputo
  - En la clase 1 se prohibía el salto y se reponía el **while**
  - Y con la autoaplicación va a pasar lo mismo, cuando veamos **fix**
- **Acá no:** la recursión estructural puede menos que la general
- Toda función estructural **termina siempre**
  - Sobre valores **totalmente definidos**: sobre uno infinito no hay dónde parar
  - Así que no puede expresar ninguna función parcial
    - Siempre que los parámetros del esquema estén totalmente definidos
- Caso concreto: un intérprete de un lenguaje con **while**
  - El programa interpretado puede no terminar (**while (true) {}**)
  - Entonces el intérprete **tiene que** poder no terminar
  - Y por eso no se escribe con un fold
  - Vamos a ver intérpretes en clases posteriores

## Entonces, ¿para qué la queremos?

- Porque disciplina, y eso vale por sí solo
- Porque la enorme mayoría de las funciones **sí** entran en el esquema
- Porque cuando entran, se gana todo lo que sigue
- Y la recursión general **queda a mano** para los casos que la necesitan
- No es "usar solo esquemas"
  - Es **usar el esquema cuando se puede, y saber por qué cuando no**
- El argumento no es nuevo: **John Backus** lo hizo en 1977

# John Backus

Fue un científico de la computación estadounidense que trabajó toda su carrera en IBM. Dirigió el equipo que creó **FORTRAN** en 1957, el primer lenguaje de alto nivel de uso masivo, e introdujo la notación que lleva su nombre (*BNF*) para describir la sintaxis de ALGOL 60. En 1977, al recibir el premio Turing, dio una conferencia titulada *¿Puede liberarse la programación del estilo von Neumann?*, donde sostuvo que el problema de los lenguajes convencionales es que no tienen leyes útiles para razonar sobre los programas, y propuso un estilo funcional armado con un puñado de formas de combinación. Una de ellas es la reducción, que es exactamente un plegado.



3 de diciembre de 1924 –  
17 de marzo de 2007

## Y lo que se compra

- Una recursión general puede no terminar
  - Y es mucho más difícil razonar sobre ella
- Una recursión **estructural** siempre termina sobre valores **finitos**
  - Que son los que la inducción cubre
  - El argumento se achica en cada paso, hasta un constructor base
- Y admite un principio de prueba: la **inducción estructural**
- Ese es el tema de la próxima clase
- El esquema no es prolijidad: es lo que hace demostrable al programa

## Lo que vimos

- El esquema **foldr** sale de recuadrar lo que cambia
  - Y reemplaza los constructores de la lista
- Los esquemas **map** y **filter** son instancias de **foldr**
- **Recursión estructural**: la forma de definir que **foldr** captura
- El esquema **recr**, para cuando hace falta la estructura original
- El plegado **foldl**: recursión a la cola, acumulando desde el principio
- Los **teoremas de dualidad**: cuándo coinciden **foldr** y **foldl**
  - Y el tercero, que son el mismo plegado dado vuelta

# Lo que hay que llevarse

- La **forma del tipo determina el esquema**

- Y por eso hay un fold para cada tipo algebraico

- La recursión estructural **sí** pierde poder,  
y por eso la general queda a mano

# Una nota sobre el método

## ■ Esta clase siguió un recorrido, y no es casual

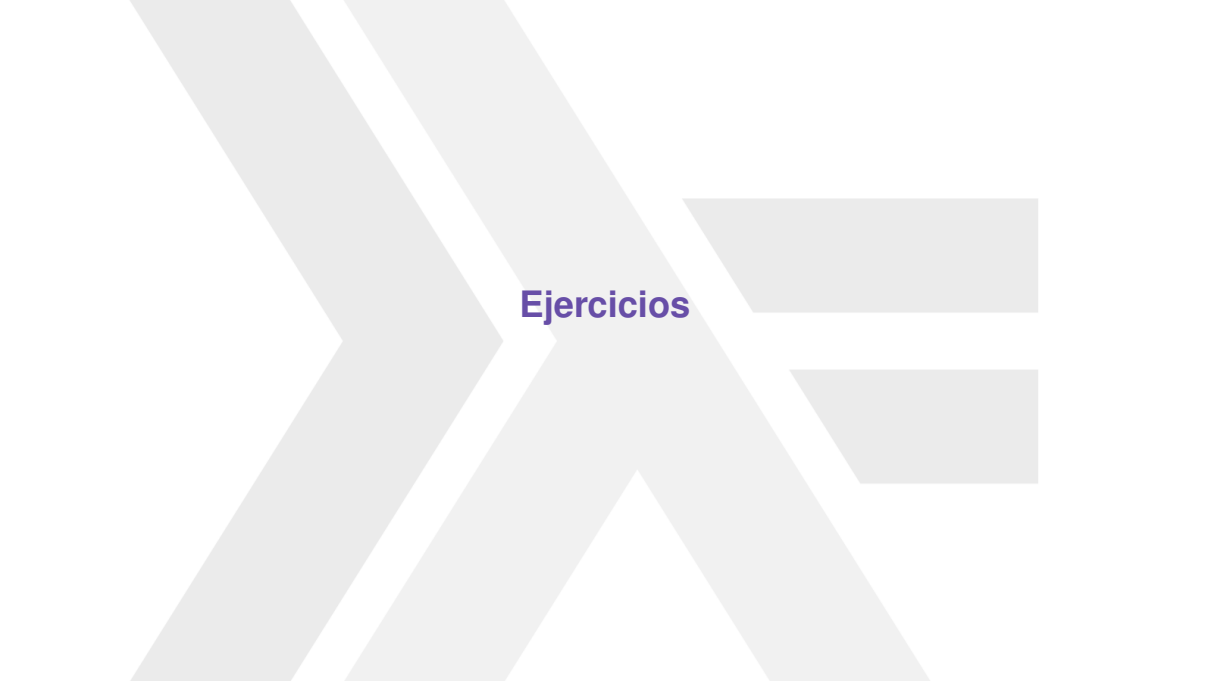
1. Qué problema la motiva, y qué es
2. Cómo se usa
3. **Cuándo** se puede usar: la condición exacta, no la intuición
4. Qué queda **afuera**, y qué condición se rompe ahí
5. Con qué se reemplaza en esos casos

- ☐ A veces se ve en el momento, a veces solo se nombra

## ■ El paso 4 no es un apéndice: es **parte de la técnica**

- ☐ Una técnica que sirve para todo no restringe nada
- ☐ Y lo que se gana sale siempre de la restricción

## ■ Saber dónde deja de servir es parte de saber usarla



## Ejercicios



# Reconocer el esquema

## ■ ¿Cuáles de estas son estructurales?

- ☐ Para las que sí, escribirlas con **foldr**
- ☐ Y para las que no, ver si se pueden hacer con **recr**
- ☐ ¿Alguna pide **foldl**?

```
último [x] = x
último (x : xs) = último xs
```

```
reverso [] = []
reverso (x : xs) = reverso xs ++ [x]
```

```
trim [] = []
trim (x : xs) = if x == ' ' then trim xs else x : xs
```

```
bin2dec' ac [] = ac
bin2dec' ac (b : bs) = bin2dec' (b + 2 * ac) bs
```

## Para pensar

```
zip :: [a] -> [b] -> [(a,b)]
zip [] = \ys -> []
zip (x : xs) = \ys ->
 case ys of
 [] -> []
 (y : zs) -> (x, y) : zip xs zs
```

- Si **foldr** **(:)** **[]** es la identidad, ¿qué es **foldl** **(flip (:))** **[]**?
  - Pista: **foldl f z [x1, x2, x3] = f (f (f z x1) x2) x3**
- Verificar las hipótesis del 2do teorema para **postfix** y **prefix**
  - Y concluir **reverse = rev**
- ¿Vale el 1er teorema para **(++)** y **[]**? ¿Y para **(-)** y **0**?
- Escribir **zip** con **foldr**
  - ¿Sobre cuál de las dos listas se pliega, y qué describe el plegado?

# Plegar árboles

```
data AB a = Nil | Bin (AB a) a (AB a)
foldAB :: b -> (b -> a -> b -> b) -> AB a -> b
foldAB z g Nil = z
foldAB z g (Bin ti x td) = g (foldAB z g ti) x (foldAB z g td)
```

## ■ Con **foldAB**, definir

```
posorden :: AB a -> [a]
todosLosCaminos :: AB a -> [[a]]
mapAB :: (a -> b) -> AB a -> AB b
```

- ☐ La función **posorden** da los nodos, primero los dos subárboles y al final la raíz
- ☐ Y **todosLosCaminos** da los caminos de la raíz a una hoja
  - ☐ ¿Qué tipo tiene acá la **b** del esquema?
- ☐ ¿Alguna necesita el subárbol **original**, y no su plegado?

## Para pensar: árboles

- ¿Qué función es `foldAB Nil Bin`?
- ¿Se puede escribir `insertarABB` con `foldAB`?

```
insertarABB :: Ord a => a -> AB a -> AB a
insertarABB x Nil = Bin Nil x Nil
insertarABB x (Bin ti y td) =
 if x < y
 then Bin (insertarABB x ti) y td
 else Bin ti y (insertarABB x td)
```

- ☐ Es la misma pregunta que `insertarOrdenado` sobre listas
- ☐ ¿Cómo sería el esquema de recursión primitiva de los árboles?

## Aplicar la receta

- Dado este tipo, escribir su esquema de recursión

```
data Expr = Lit Int
 | Suma Expr Expr
 | Prod Expr Expr
```

- Con ese esquema, definir **evaluar** y **cantLits**
  - Las dos de tipo **Expr -> Int**
- ¿Cuántos parámetros tiene el esquema, y por qué?

## Para leer

- Graham Hutton. *A tutorial on the universality and expressiveness of fold*
  - Journal of Functional Programming 9(4): 355–372, julio de 1999
- Peter A. Fejer y Dan A. Simovici. *Mathematical Foundations of Computer Science, Vol. 1: Sets, Relations, and Induction*
  - Springer, 1991. Capítulo 4, *Induction*